

Container and pod management

Running containers: Like Docker, Podman allows you to manage containers. You can list running containers, start/stop containers, and remove them using familiar commands.

- *Listing containers:*

```
podman ps -a
```

- *Starting a container:*

```
podman start <container_id>
```

- *Stopping a container:*

```
podman stop <container_id>
```

Pod management: With Podman, you can also create and manage pods, which are groups of containers that share the same resources. Here's how to create a pod and run containers within it:

```
podman pod create --name mypod
podman run -d --pod mypod nginx
podman run -d --pod mypod redis
```

In this example, Nginx and Redis containers are created within the same pod, allowing them to share network and storage resources seamlessly.

Applications

Podman is widely used in various development, testing, and deployment phases. Some common use cases include:

Development and testing:

Developers can run and test containers locally without root privileges. This reduces the risk of security breaches and ensures compatibility with production environments.

CI/CD pipelines: Podman can be integrated into CI/CD pipelines to automate the building, testing, and deployment of containers, providing flexibility for different stages of the DevOps lifecycle.

Kubernetes integration: Since Podman supports Kubernetes pods, it can be used in the development phase of

Table 1: Podman and Docker: A comparison

Feature	Podman	Docker
Architecture	Daemon-less	Centralised daemon
Rootless operation	Supports rootless containers	Requires root privileges for containers
Pod support	Native support for Kubernetes-style pods	No direct pod support
Security	Enhanced security with rootless containers	Daemon presents a security risk
Kubernetes integration	Native pod concept for a smooth transition	Requires additional setup
Compatibility	OCI-compliant and Docker-compatible CLI	Docker-specific commands

Kubernetes applications. You can define your containers and pods locally using Podman and easily transition them to Kubernetes clusters.

Podman vs Docker

While Podman and Docker facilitate containerisation, they differ in architecture and security. Podman's daemon-less and rootless design offers better security by running containers without requiring root access, reducing potential vulnerabilities. On the other hand, Docker relies on a central daemon that must run as root, presenting security risks but offering simplicity through a single management service (see Table 1).

Podman also aligns more closely with Kubernetes through its support for pods, a core feature of Kubernetes. However, Docker requires third-party tools like Docker Compose to manage multi-container setups. Docker's

well-established ecosystem, including Docker Hub and Docker Swarm, makes it the go-to solution for many container deployments.

In summary, Podman excels in security-conscious environments and Kubernetes-native workflows. In contrast, Docker's maturity and ease of use make it ideal for developers seeking simplicity and extensive third-party integrations.

Podman is increasingly becoming a valuable tool for container management, offering unique advantages over Docker with respect to security, flexibility, and Kubernetes integration. Its rootless and daemon-less features will likely gain even more attention as organisations seek to improve security practices and avoid root-level vulnerabilities.

Whether you're a developer, system administrator, or someone working in CI/CD, Podman is worth exploring for its cutting-edge capabilities in containerisation. **END** 🐧

References

- <https://www.redhat.com/en/topics/containers/what-is-podman>
- <https://podman.io/>
- <https://minikube.sigs.k8s.io/docs/drivers/podman/>
- <https://www.keycloak.org/getting-started/getting-started-podman>
- Kang, H., & Barnes, J. (2020), Podman: A next-generation container engine, IBM Developer; <https://developer.ibm.com/articles/podman-next-gen-container-engine/>

By: Dulari Bhatt and Dhanashree Yevle

Dulari Bhatt is master trainer at Edunet Foundation and has around 12 years of academic experience. Her core expertise is in computer vision, machine learning, deep learning, and data analytics.

Dhanashree Yevle is a research scholar specialising in artificial intelligence, machine learning, and deep learning.